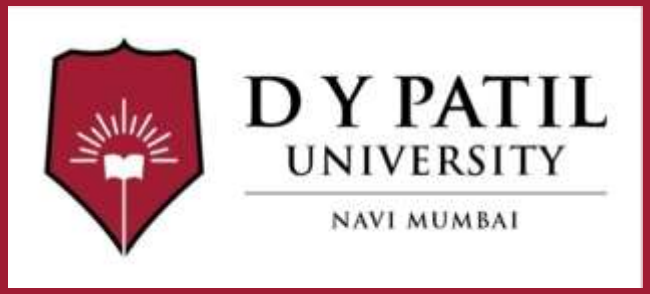
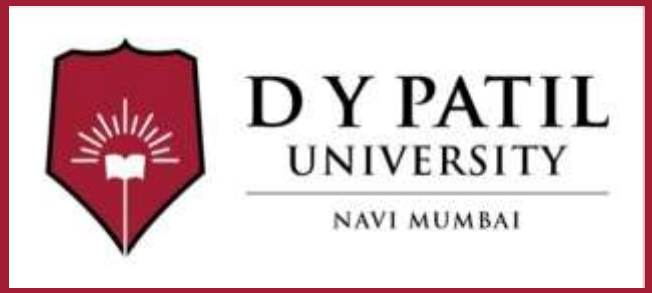


Subject Name :Web Development with Java

Unit No:V

Unit Name :Data Persistence, Transactions, and Testing with Spring Boot

Faculty Name :Dr. Saloni Bhushan



Unit No.5

Faculty Name : Dr. Saloni Bhushan

Data Persistence with JPA and Hibernate

- Introduction to ORM and JPA
- Hibernate as JPA implementation
- Entity mapping using annotations
- CRUD operations using Spring Data JPA
- Repository interfaces

Transaction Management and Exception Handling

- Need for transaction management
- Transaction management using `@Transactional`
- Exception handling in RESTful services
- Global exception handling using `@ControllerAdvice`
- Custom exception handling and HTTP status codes

Testing Spring Boot Applications

- Introduction to testing in Spring Boot
- Unit testing using Junit
- JUnit lifecycle
- Testing REST controllers
- Importance of automated testing

D Y PATIL
UNIVERSITY
NAVI MUMBAI

Lecture No: 1

Introduction to ORM and JPA

ORM (Object Relational Mapping) is a technique used to map Java objects to database tables.

- In Java, data is represented as **objects**
- In relational databases, data is stored in **tables**
- ORM acts as a bridge between the **object-oriented world** and the **relational database world**

Example:

A Java class:

```
class Student {
   int id;
   String name;
}
```

can be mapped to a database table:

id	name
1	John

Why ORM

Without ORM, developers need to write SQL queries manually:

INSERT INTO student VALUES(1,'John');

With ORM, developers work with Java objects:

studentRepository.save(student);

Benefits

Reduces boilerplate SQL code
Improves productivity
Makes applications database-independent
Simplifies CRUD operations
Better maintainability

What is JPA?

JPA (Java Persistence API) is a Java specification for managing relational data in Java applications.

It provides:

- Rules for mapping Java classes to database tables
- APIs for CRUD operations
- Standard annotations for ORM

JPA is **not an implementation**, it is only a **specification**.

JPA Implementations

To use JPA, we need an implementation such as:

- **Hibernate**
- **EclipseLink**
- **OpenJPA**

Among these, **Hibernate** is the most popular.

Flow:

Java Application → JPA → Hibernate → Database

Key JPA Annotations

These annotations come from the Java Persistence API.”
import jakarta.persistence.Entity;

@Entity

Marks a Java class as an entity.

```
@Entity
public class Student {
}
```

@Table

Maps the entity to a specific table.

```
@Table(name="student")
```

@Id

Specifies the primary key.

```
@Id
private int id;
```

@Column

Maps a field to a table column.

```
@Column(name="student_name")
private String name;
```

Example Entity Class

```
import jakarta.persistence.*;

@Entity
@Table(name="student")
public class Student {

   @Id
   private int id;

   @Column(name="name")
   private String name;
}
```

This maps the `Student` class to the `student` table.

JPA in Spring Boot

In a Spring Boot application, we need a way to interact with the database for operations such as:

- inserting data
- retrieving data
- updating records
- deleting records

Traditionally, this required writing SQL queries manually.

To simplify this, Spring Data JPA provides repository interfaces with built-in methods for database operations.

Repository Interface

A repository interface acts as the **data access layer** between the application and the database.

In Spring Data JPA, JpaRepository is an interface that provides **ready-made methods** to perform database operations.

It is used to interact with the database without writing SQL queries manually.

In Spring Boot, we create a repository interface by extending JpaRepository.

Example:

```
public interface StudentRepository extends JpaRepository<Student, Integer> {
}
```

- `Student` → the entity class
- `Integer` → the primary key type

By extending `JpaRepository`, Spring Data JPA automatically provides methods such as:

- `save()` → insert or update data
- `findAll()` → retrieve all records
- `deleteById()` → delete a record by ID

So the developer can perform CRUD operations **without writing SQL queries manually**.

What is Hibernate?

Hibernate is an **ORM (Object Relational Mapping)** framework for Java.

It is used to map:

- **Java classes → Database tables**
- **Java objects → Database records**

Hibernate reduces the need to write SQL queries manually and simplifies database operations.

Hibernate as JPA Implementation

JPA (Java Persistence API) is only a **specification** that defines rules for ORM.

It does not provide actual implementation.

Hibernate provides the actual implementation of JPA interfaces and annotations.

Relationship:

JPA Specification → Hibernate Implementation → Database

This means:

- JPA defines the standards
- Hibernate follows these standards and performs database operations

Why Hibernate?

Hibernate is widely used because it:

- Implements JPA standards
- Reduces boilerplate JDBC code
- Automatically generates SQL queries
- Handles database connections internally
- Supports relationships between entities
- Improves productivity

Hibernate Architecture

Hibernate works between the Java application and the database.

Java Application

↓

JPA

↓

Hibernate

↓

Database

The application uses JPA annotations, and Hibernate converts them into SQL queries.

Example of Hibernate as JPA Implementation

Entity Class

```
import jakarta.persistence.*;
@Entity
@Table(name="student")
public class Student {
  @Id
  private int id;
  private String name;
}
```

This class uses **JPA annotations**, but Hibernate executes the actual SQL operations.

Example Repository in Spring Boot

```
public interface StudentRepository extends
JpaRepository<Student, Integer> {
}
```

When we call:

```
studentRepository.save(student);
```

Hibernate generates SQL like:

```
INSERT INTO student (id, name) VALUES (?, ?);
```

So the developer writes Java code, and Hibernate handles SQL generation.

Hibernate Features

1. Automatic Table Mapping

Maps entity classes to database tables using annotations.

Example:

- @Entity
- @Table
- @Id

2. Automatic SQL Generation

Hibernate generates SQL queries for:

- Insert
- Update
- Delete
- Select

3. Caching Support

Hibernate improves performance using:

- First-level cache
- Second-level cache

This reduces repeated database access.

Hibernate Features

4. Relationship Mapping

Hibernate supports relationships such as:

- @OneToOne
- @OneToMany
- @ManyToOne
- @ManyToMany

5. Database Independence

Hibernate works with different databases like:

- MySQL
- PostgreSQL
- Oracle

Changing the database requires minimal code changes.

Hibernate in Spring Boot

In Spring Boot, Hibernate is the **default JPA provider**.

When we add:

spring-boot-starter-data-jpa

Spring Boot automatically configures:

- JPA
- Hibernate
- Data source

This makes development easier.

Advantages of Hibernate as JPA Provider

Easy integration with Spring Boot
Less JDBC code
Portable across databases
Automatic schema generation
Better maintainability
Faster development

Example Application

This example uses:

- @Entity
- JpaRepository
- Controller
- CRUD methods:
  - save()
  - findAll()
  - findById()
  - deleteById()

application.properties

Required Dependencies for Your Project

1. Spring Web

For creating REST APIs.

2. Spring Data JPA

For database operations using JPA.

Spring Data JPA

3 For connecting Spring Boot to MySQL.

MySQL Driver

. Entity Class

```
package com.example.demo.entity;
import jakarta.persistence.Entity;
import jakarta.persistence.Id;
@Entity
public class Student {

    @Id
    private int id;
    private String name;
    public Student() {
    }
    public Student(int id, String name, String course) {
        this.id = id;
        this.name = name;
    }
    public int getId() {
        return id;
    }
    public void setId(int id) {
        this.id = id;
    }
    public String getName() {
        return name;
    }
    public void setName(String name) {
        this.name = name;
    }
    @Override
    public String toString() {
        return id + " " + name + " " + course;
    }
}
```

Repository Interface `StudentRepository.java`

```
package com.example.demo.repository;

import org.springframework.data.jpa.repository.JpaRepository;
import com.example.demo.entity.Student;

public interface StudentRepository extends JpaRepository<Student, Integer> {

}
```

Common methods of JPARepository

Method	Purpose
`save()`	Insert / Update
`findAll()`	Retrieve all
`findById()`	Retrieve one
`deleteById()`	Delete one

3. Controller Class

```
package com.example.demo.controller;

import java.util.List;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;

import com.example.demo.entity.Student;
import com.example.demo.repository.StudentRepository;

@RestController
@RequestMapping("/students")
public class StudentController {

    @Autowired
    private StudentRepository repo;

    // CREATE
    @PostMapping
    public Student addStudent(@RequestBody Student student) {
        return repo.save(student);
    }

    // READ ALL
    @GetMapping
    public List<Student> getAllStudents() {
        return repo.findAll();
    }

    // READ BY ID
    @GetMapping("/{id}")
    public Student getStudentById(@PathVariable int id) {
        return repo.findById(id).orElse(null);
    }

    // UPDATE
    @PutMapping
    public Student updateStudent(@RequestBody Student student) {
        return repo.save(student);
    }

    // DELETE
    @DeleteMapping("/{id}")
    public String deleteStudent(@PathVariable int id) {
        repo.deleteById(id);
        return "Student deleted successfully";
    }
}
```

```
// CREATE
        Student s1 = new Student(101, "John", "MCA");
        repo.save(s1);
        System.out.println("Student inserted");

        // READ ALL
        System.out.println("All Students:");
        repo.findAll().forEach(System.out::println);

        // READ BY ID
        System.out.println("Student with ID 101:");
        System.out.println(repo.findById(101).orElse(null));

        // UPDATE
        Student s = repo.findById(101).orElse(null);
        if (s != null) {
            s.setCourse("MBA");
            repo.save(s);
            System.out.println("Student updated");
        }

        // DELETE
        repo.deleteById(101);
        System.out.println("Student deleted");
    }
}
```

4. Database Configuration

application.properties

```
spring.datasource.url=jdbc:mysql://localhost:3306/testdb
spring.datasource.username=root
spring.datasource.password=root

spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
```

application.properties

1. Database URL

spring.datasource.url=jdbc:mysql://localhost:3306/testdb

2. Username and Password

spring.datasource.username=root
spring.datasource.password=root

3. `ddl-auto=update`

spring.jpa.hibernate.ddl-auto=update

> "Hibernate automatically creates or updates the database table based on the entity class."

`Student.java` → table creation happens automatically

4. `show-sql=true`

spring.jpa.show-sql=true

> "This displays SQL queries generated by Hibernate in the console.

Expected Output

Student inserted

All Students:
101 John MCA

Student with ID 101:
101 John MCA

Student updated
Student deleted

Transaction Management

A **transaction** is a sequence of one or more database operations treated as a **single unit of work**.

A transaction ensures that either:

- **All operations are completed successfully**, or
- **None of them are applied**

This protects data consistency.

Example

Suppose money is transferred from one bank account to another:

1. Deduct amount from Account A
2. Add amount to Account B

If the first step succeeds but the second fails, the database becomes inconsistent.

Transaction Management using `@Transactional`

In Spring Framework, transaction management is handled using the **`@Transactional`** annotation. This annotation ensures that a method executes within a transaction.

Syntax

```
@Transactional
public void processOrder() {
   ...
}
```

When a method is marked with **`@Transactional`**:

1. Spring starts a transaction
2. Executes all database operations
3. If successful → commits transaction
4. If exception occurs → rolls back transaction

Example

```
@Transactional
public void transferMoney() {
   accountRepository.withdraw();
   accountRepository.deposit();
}
```

If **`deposit()`** fails, **`withdraw()`** is rolled back automatically.

Exception Handling in RESTful Services

In REST APIs, errors may occur due to:

- invalid input
- resource not found
- database errors
- server errors

Instead of crashing the application, the API should return **meaningful error responses**.

Example

If a student is not found:

Instead of: NullPointerException

Return:

404 Not Found

This improves API usability.

Common HTTP error

Status Code	Meaning
400	Bad Request
404	Not Found
500	Internal Server Error

Global Exception Handling using `@ControllerAdvice`

In Spring Framework, **@ControllerAdvice** is used to handle exceptions globally for all controllers.

Instead of writing exception handling code in every controller, we define it in one central class.

```
@ControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(Exception.class)
    public String handleException(Exception ex) {
        return ex.getMessage();
    }
}
```

Custom Exception Handling and HTTP Status Codes

Sometimes we need application-specific exceptions such as:

- Student not found
- Invalid account
- Order not found

For this, we create **custom exceptions**.

Custom Exception Example

```
public class StudentNotFoundException extends RuntimeException {
    public StudentNotFoundException(String message) {
        super(message);
    }
}
```

This exception can be thrown when a student record is missing.

Throwing the Exception

```
throw new StudentNotFoundException("Student not found");
```

Handling Custom Exception

```
@ExceptionHandler(StudentNotFoundException.class)
@ResponseStatus(HttpStatus.NOT_FOUND)
public String handleStudentNotFound(StudentNotFoundException ex) {
    return ex.getMessage();
}
```

@ExceptionHandler

Specifies which exception to handle.

@ResponseStatus

Sets the HTTP status code returned to the client.

Result

If student is not found:

Response:

Student not found

HTTP Status:

404 Not Found

Testing Spring Boot Applications

Testing is the process of verifying whether an application works as expected.

In Spring Boot, testing is used to check:

- whether application components work correctly
- whether APIs return expected responses
- whether bugs are detected early

Spring Boot provides built-in support for testing using **JUnit** and Spring testing tools.

Unit Testing using JUnit

Unit testing is the process of testing the **smallest unit of code**, usually a method.

In Java, unit testing is commonly done using JUnit.

Each test method checks one specific functionality.

Example

Application method:

```
public int add(int a, int b) {
    return a + b;
}
```

Test method:

```
@Test
public void testAdd() {
   assertEquals(5, add(2,3));
}
```

@Test

Marks a method as a test method.

assertEquals()

Checks whether the expected output matches the actual output.

Benefits of unit testing

tests methods independently
finds bugs quickly
improves reliability

JUnit Lifecycle

JUnit provides lifecycle annotations to control the execution of test methods.

These annotations define what should happen:

- before all tests
- before each test
- after each test
- after all tests

JUnit Lifecycle Annotations

@BeforeAll

Runs once before all test methods.

```
@BeforeAll
static void setup() {
   System.out.println("Before all tests");
}
```

@BeforeEach

Runs before each test method.

```
@BeforeEach
void init() {
   System.out.println("Before each test");
}
```

@AfterEach

Runs after each test method.

```
@AfterEach
void cleanup() {
   System.out.println("After each test");
}
```

@AfterAll

Runs once after all test methods.

```
@AfterAll
static void finish() {
   System.out.println("After all tests");
}
```

Lifecycle Order

@BeforeAll
@BeforeEach
@Test
@AfterEach
...
@AfterAll

Testing REST Controllers

REST controller testing checks whether API endpoints return the correct response.

In Spring Boot, this is done using:

- @SpringBootTest
- @AutoConfigureMockMvc
- MockMvc

Example Controller

```
@GetMapping("/hello")
public String hello() {
  return "Hello";
}
```

Test Case

```
@SpringBootTest
@AutoConfigureMockMvc
public class ControllerTest {

  @Autowired
  private MockMvc mockMvc;

  @Test
  public void testHello() throws Exception {
     mockMvc.perform(get("/hello"))
          .andExpect(status().isOk())
          .andExpect(content().string("Hello"));
  }
}
```

MockMvc

Used to test HTTP requests without running the actual server.

status().isOk()

Checks whether status code is 200 OK.

content().string()

Checks the returned response body.

Importance of Automated Testing

Automated testing means running tests automatically using test cases.

Instead of checking functionality manually, tests verify the application behavior automatically.

Benefits

1. Saves Time

Repeated tests can run automatically.

2. Improves Reliability

Reduces human error.

3. Detects Bugs Early

Errors are found during development.

4. Supports Code Changes

After modifying code, tests ensure existing functionality still works.

5. Improves Software Quality

Provides confidence that the application works correctly.

Example

If we modify a controller method, automated tests verify whether the API still returns the expected output.

This prevents accidental errors.